

Graph-Based Vulnerability Retrieval

Master's Thesis — Technical Progress · Checkpoint 4

Lívia

TU Munich · Siemens

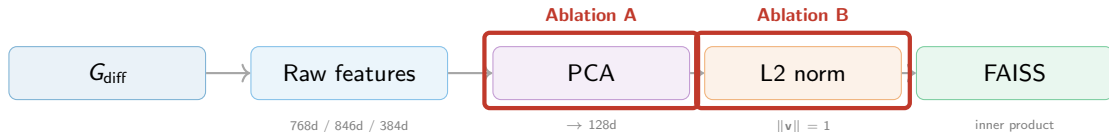
May 4, 2026

Agenda

- 🔄 **Checkpoint 3 recap** — results from slicing ablation & embedding pipeline ablation &
- ✂️ **Slicing ablation** — full graph vs. sliced, with/without diff labels
- 🏗️ **CWE ontology** — structure & Wu-Palmer similarity
- 🔗 **Cross-CVE callgraph** — shared callee signatures as a second signal
- 📖 **Knowledge-augmented reranking** — embedding + Jaccard fusion
- 📊 **Embedder ensembles** — majority vote, cascade, RRF strategies
- 🔍 **Combination strategies** — concat→PCA vs. PCA→concat→PCA vs. norm→concat→PCA
- 🔴 **Confusion analysis** — where every embedder still fails
- 💡 **Key findings & next steps**

From Last Week

Combined Embedding Pipeline: Where PCA & L2 Normalization Sit



PCA (dimensionality reduction)

- Projects high-dim features to 128d
- Learns optimal linear basis from data
- Decorrelates and compresses — fuses heterogeneous streams (e.g. 78d structural + 768d CodeBERT)

L2 normalization (scale invariance)

- Maps every embedding onto the unit hypersphere
- FAISS inner product = cosine similarity
- Similarity depends on **direction** (pattern), not **magnitude** (graph size)

Ablation Results: PCA and L2 Normalization ($n=225$, self-retrieval)

Embedder	PCA + L2 (baseline)			No PCA, L2 only (Abl. A)			No PCA, no L2 (Abl. B)		
	hit@1	MRR	CWE	hit@1	MRR	CWE	hit@1	MRR	CWE
Combined	84.9%	.890	.501	74.0%	.780	.398	1.4%	.047	.071
CodeBERT seq	75.3%	.793	.412	76.7%	.799	.402	1.4%	.107	.160
GIN	72.6%	.788	.443	72.6%	.788	.443	1.4%	.044	.068
CodeBERT pat.	71.2%	.768	.414	75.3%	.788	.394	1.4%	.132	.201
WL	65.8%	.720	.423	67.1%	.723	.435	0.0%	.045	.065
RGCN	60.3%	.676	.372	67.1%	.733	.390	0.0%	.014	.060

Baseline: PCA + L2

- Combined dominates: 84.9%
- PCA is critical for Combined (−10.9 pp without)

Abl. A: No PCA

- CodeBERT stable (± 1 pp)
- GIN identical — already 128d
- Combined collapses: PCA fuses the 3 sub-embedders

Abl. B: No L2

- **Catastrophic failure** — all $< 2\%$
- Mean sim $> 100k$ (magnitude dominates)
- L2 norm is non-negotiable

Embedding Space Analysis: Metrics

Single-space quality (per embedder)

Metric	What it measures
Isotropy	Are embeddings spread uniformly across directions? Low $\rightarrow$ a few dimensions dominate $\rightarrow$ retrieval degrades.
Hubness	Do some points appear as neighbors of <i>everyone</i> ? High skewness $\rightarrow$ false-positive retrievals.
Alignment / Uniformity	Alignment: same-CWE pairs are close. Uniformity: all points spread on hypersphere. Decomposes retrieval quality into two axes.
Intra/Inter ratio	$\frac{\text{mean intra-CWE sim}}{\text{mean inter-CWE sim}}$. >1 means classes separated $\rightarrow$ good CWE recall.
Distance concentration	$\frac{d_{\max} - d_{\min}}{d_{\min}}$ per query. Low $\rightarrow$ all distances alike $\rightarrow$ cosine unreliable.

Cross-space comparison (Combined vs. individual)

Metric	What it measures
Linear CKA	Structural similarity between two embedding matrices. Invariant to rotation/scaling. High $\rightarrow$ spaces encode the same geometry.
k -NN overlap	% shared neighbors between two spaces. High $\rightarrow$ same retrieval behavior (redundant). Low $\rightarrow$ complementary signal.
Rank correlation	Spearman ρ of pairwise distances. High $\rightarrow$ one embedder dominates the fusion.
Trustworthiness	Did PCA (384d $\rightarrow$ 128d) distort local neighborhoods? 1.0 = perfect preservation. Low $\rightarrow$ PCA causes retrieval errors.

🎯 Why these metrics?

- Diagnose *why* Combined outperforms individuals
- Identify which sub-embedder dominates the PCA
- Detect degenerate spaces *before* running retrieval
- Explain per-query errors via space properties

Embedding Space Analysis: Experiment Design

Setup (same split as retrieval experiments)

- 225 index graphs (G_{diff}), seed=42, stratified
- 6 embedders: Combined, GIN, WL, CodeBERT seq, CodeBERT pat., RGCN
- All projected to 128d (PCA) + L2-normalized
- CWE labels as class signal (~ 15 unique classes)

Analysis structure (no grid — fixed dataset)

Analysis	Output
Per-embedder	Isotropy, hubness, alignment/uniformity, intra/inter, distance concentration
Pairwise matrices	6×6 CKA, k -NN overlap, rank correlation
Combined PCA	Trustworthiness at $k \in \{5, 10, 20\}$
Sensitivity	Hubness skewness vs. k (stability check)

Key questions this answers

- 1 **Does Combined add value?**
If $\text{CKA}(\text{Combined}, \text{GIN}) \approx 1 \rightarrow \text{Combined} \approx \text{GIN}$ with extra cost.
If k -NN overlap is low $\rightarrow$ Combined retrieves differently.
- 2 **Which sub-embedder dominates?**
Rank correlation identifies whose distance structure is preserved in the fusion.
- 3 **Is PCA lossy?**
Trustworthiness $< 0.9 \rightarrow$ PCA distorts neighborhoods $\rightarrow$ explains Combined-specific misses.
- 4 **Why do some CWEs fail?**
Per-CWE intra/inter ratio shows which classes overlap in each space.
- 5 **Are there hub points?**
Hub fraction $> 5\% \rightarrow$ some samples “attract” queries $\rightarrow$ systematic false positives.

Run: `python -m experiments.space_analysis_experiment`

Combination Strategies: Experiment Design

Goal

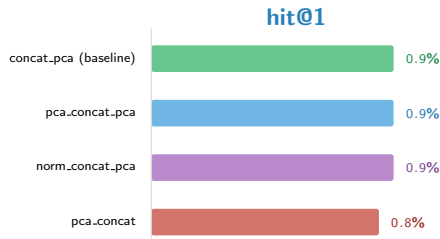
The Combined embedder concatenates NetLSD + WL + GIN then reduces with PCA. Does pre-processing each sub-embedder *before* concatenation improve the fused space?

Strategies compared

Strategy	Pipeline
concat_pca	Concatenate raw $\rightarrow$ PCA to 128d (baseline)
pca_concat_pca	PCA each to 128d $\rightarrow$ concat (384d) $\rightarrow$ PCA to 128d
pca_concat	PCA each to 42d $\rightarrow$ concat to 126d (no 2 nd PCA)
norm_concat_pca	L2-norm each $\rightarrow$ concat $\rightarrow$ PCA to 128d

Rationale

Individual PCA or normalization could equalize sub-embedder scales and remove noise before fusion. Alternatively, a single final PCA may better exploit cross-embedder correlations.



225 index, 73 query (G_{diff} , seed=42).
Each sub-embedder: 128d (NetLSD, WL, GIN).
Run: `python -m experiments.combining_experiment`

Combination Strategies: Results ($n=73$)

Strategy	Retrieval						Space Quality		
	hit@1	hit@5	hit@10	MRR	nDCG@10	MAP@10	Eff. Dim	Isotropy	Hub Skew
concat_pca	89.0%	95.9%	97.3%	.915	.687	.567	30.9	.0024	1.40
pca_concat_pca	89.0%	95.9%	97.3%	.915	.687	.567	30.9	.0023	1.40
norm_concat_pca	89.0%	95.9%	97.3%	.915	.687	.567	30.9	.0024	1.40
pca_concat	83.6%	95.9%	97.3%	.882	.665	.538	23.0	≈ 0	1.09

Baseline is optimal

- Three strategies are **identical** to 4+ decimal places
- Sub-embedders already output 128d L2-normed vectors $\rightarrow$ pre-processing is a no-op

pca_concat hurts

- hit@1 drops -5.4 pp, MRR -3.3 pp
- PCA to 42d per embedder loses too much information
- Isotropy collapses (≈ 0): space degenerate

Why single PCA wins

- Final PCA over full 384d can exploit **cross-embedder correlations**
- Independent per-embedder PCA cannot discover joint structure
- Eff. dim 30.9 vs. 23.0: richer space

Combination Strategies: Key Findings

- ✓ **Current pipeline is already well-conditioned.** Pre-normalizing or pre-reducing individual embeddings before concatenation adds zero measurable benefit — the sub-embedders already produce 128d L2-normalized vectors. *No pipeline change needed.*
- ✗ **Double PCA without final reduction = information loss.** Reducing each embedder to 42d then concatenating (126d, no second PCA) drops hit@1 by 5.4 pp and collapses isotropy to ≈ 0 . *The final PCA is essential — it finds the best 128 directions across the full 384d concatenated space.*
- 📊 **Cross-embedder correlations matter.** A single PCA over the joint space (eff. dim 30.9) outperforms three independent PCAs (eff. dim 23.0) because it captures shared structure between NetLSD, WL, and GIN.
- 💡 **Implication for future embedders.** When adding new sub-embedders (e.g. CodeBERT), they should be concatenated raw into the joint space and let PCA handle fusion — no per-embedder preprocessing is warranted.

4-Way Fusion: Experiment Design

Goal

Can adding a semantic signal (CodeBERT-pattern) to the structural fusion (NetLSD + WL + GIN) improve retrieval? We test two hypotheses: (1) raw concat lets PCA discover joint directions, (2) pre-PCA'ing CodeBERT to 128d equalises scale.

Run 1: raw concatenation

CodeBERT-pattern emits 802d (34 pattern + 768 CodeBERT). Concat with 3×128 d structural = 1186d $\rightarrow$ PCA to 128d.

Problem: CodeBERT contributes 68% of dimensions.

Run 2: balanced pre-PCA

PCA CodeBERT-pattern 802d $\rightarrow$ 128d first, then concat 4×128 d = 512d $\rightarrow$ PCA to 128d.

Each sub-embedder gets 25% of the pre-PCA space.

Rationale

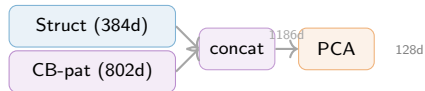
CodeBERT captures code *semantics*; structural embedders capture vulnerability *patterns*. Low CKA between them in prior experiments suggests

Pipeline comparison

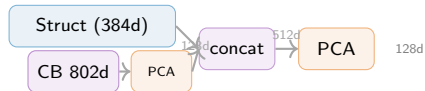
3-way (baseline)



4-way Run 1



4-way Run 2



4-Way Fusion: Results

Strategy	Retrieval				Space Quality			
	hit@1	hit@5	MRR	nDCG@10	Eff. Dim	Isotropy	Mean Sim	Intra/Inter
3-way baseline	90.4%	95.9%	.928	.690	31.2	.0023	.030	2.30
4-way raw concat (Run 1: 1186d→128d)	74.0%	86.3%	.786	.572	19.3	.0029	−.004	−6.46
4-way pre-PCA (Run 2: 512d→128d)	74.0%	86.3%	.786	.572	19.2	.0028	−.004	−6.59

CodeBERT hurts: −16 pp

- hit@1 drops from 90.4% to 74.0%
- MRR drops from .928 to .786
- Both runs produce **identical** degradation

Pre-PCA doesn't help

- Equalising dimensions (512d) gives same result as raw (1186d)
- Problem is **signal conflict**, not scale imbalance
- CodeBERT dominates PCA regardless

Space explains the failure

- Negative mean sim** (−0.004): embeddings anti-correlated
- Negative intra/inter** (−6.5): class structure inverted
- Eff. dim 19 vs. 31: PCA captures wrong directions

4-Way Fusion: Key Findings

- ❌ **CodeBERT-pattern is incompatible with structural fusion.** Adding it degrades every metric by 15–16 pp. The semantic signal encodes *what code does*, not *how the vulnerability manifests structurally* — these are contradictory for CWE discrimination. *The signals destructively interfere in PCA.*
- ⚖️ **Dimension balancing is not the fix.** Pre-PCA'ing CodeBERT to 128d (equal to each structural embedder) produces identical results to raw 802d concatenation. PCA's top components still reflect CodeBERT's high-variance semantic directions, not vulnerability-discriminative structure.
- 📊 **Negative intra/inter ratio is the diagnostic.** Same-CWE pairs become *less* similar than cross-CWE pairs (−6.5 ratio). Two functions with the same CWE can have very different code semantics — CodeBERT sees them as “different” and overrides the structural “same-pattern” signal.
- 💡 **If CodeBERT is to help, it needs late fusion.** Concatenation + PCA is early fusion and fails here. Alternatives: separate retrieval pipelines with score-level combination (RRF), learned gating, or reranking with CodeBERT as a second stage.

Agent Baseline: Experiment Design

Goal

Given a vulnerable function and a retrieved similar CVE, can an LLM generate a correct patch **by analogy**?

Compare *oracle* retrieval (perfect) vs. *embedding* retrieval (our pipeline).

Model & prompt

- **GPT-4o** (Azure), temperature=0.2, max_tokens=4096
- **One-shot patching by analogy**: system prompt contains the retrieved CVE's fix-list, variable/function mapping, root cause
- Assistant message shows the example's CoT + patched code
- User message provides the target function to patch
- Agent must output [CoT START]...[Patched Code END]

Retrieval modes

Mode	What it retrieves
Oracle	Same-CVE pair from index (perfect match) — upper bound on agent quality
Embedding	Top-1 from Combined embedder + FAISS (realistic pipeline)

Evaluation protocol

- Same split: 225 index, 73 query (seed=42)
- **Patch quality**: BLEU-4, token Jaccard, CodeBLEU proxy
- **Retrieval**: hit@1, MRR, CWE hit@5
- **Confidence**: precision/recall at score thresholds

Agent Baseline: Analysis & Failure Modes

Where the agent succeeds

Where the agent fails

Oracle vs. Embedding: what the gap tells us

- Patch quality **degrades with retrieval accuracy**: CBP .557 (oracle) vs. .409 (embedding), -15 pp
- When retrieval misses the correct CVE (31.5% of queries), the analogy is less relevant → lower patch quality
- Both retrieval **and** generation quality are bottlenecks

Open questions for improvement

- Future work on hybrid retrieval approaches
- Cross-validation with other CVE datasets

Repository: `graph-rag/` `python experiments/knowledge_experiment.py`